

Doel: jij bouwt een super simpele todo-app in Flutter.

- ✓ Alles in **1 bestand** (`lib/main.dart`)
- ✓ Geen database
- ✓ Kleine stapjes + veel uitleg
- ✓ Elke stap heeft een codeblok met **commentaar** waar de verandering zit

Wat ga je maken?

- Een scherm met een titel
- Een lijstje met taken
- Elke taak heeft:
 - een **checkbox**
 - een **tekst (label)**
- De checkbox kun je **aan/uit** zetten (toggle)
- De taken komen uit een **JSON-achtige datastructuur** (in Dart: een `List<Map<String, dynamic>>`)

Benodigdheden

- Flutter geïnstalleerd
- Een editor (bijv. VS Code)
- Een apparaat om op te draaien:
 - **Android emulator**, of
 - een **echte Android telefoon** (USB debugging aan), of
 - **Chrome** (voor web)

Tip: als je Flutter nog nooit hebt gedraaid: vraag je docent hoe je een emulator start.

Stap 1 — Een nieuw project starten (naam = `todo-app`)

Open een terminal (bijv. in VS Code: *Terminal* → *New Terminal*).

1. Ga naar de map waar jij je projecten bewaart.
2. Maak het Flutter project:

```
flutter create todo-app
```

Wat gebeurt er?

- Flutter maakt een map `todo-app`
- Daarin zitten veel bestanden
- Jij gaat bijna alleen werken in `**lib/main.dart**`

3. Ga de projectmap in:

```
cd todo-app
```

Stap 2 — Het project runnen in de terminal

In dezelfde terminal (in de map `todo-app`) run je:

```
flutter run
```

Wat moet je zien?

- Flutter start een app (op je emulator/telefoon of in Chrome)
- Je ziet waarschijnlijk een standaard “counter app”

Krijg je een lijst met devices (zoals Chrome/Android)?

- Kies dan het nummer van jouw device en druk op Enter.

Stap 3 — We vervangen de standaard app door een lege Todo-app

Open `**lib/main.dart**`.

Verwijder alles en plak dit:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const TodoApp());
}

class TodoApp extends StatelessWidget {
```

```

const TodoApp({super.key});

@override
Widget build(BuildContext context) {
  return const MaterialApp(
    // CHANGE: we gebruiken een eigen start-scherm i.p.v. de counter app
    home: TodoHomePage(),
  );
}

class TodoHomePage extends StatelessWidget {
  const TodoHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        // CHANGE: simpele titel
        title: const Text('Todo-app'),
      ),
      body: const Center(
        // CHANGE: we laten alleen even tekst zien
        child: Text('Hier komen straks taken!'),
      ),
    );
  }
}

```

✓ Run opnieuw:

```
flutter run
```

Je ziet nu een scherm met de titel **Todo-app**.

Stap 4 — Zelf een widget maken: Task

Wat is een widget?

In Flutter is **alles** een widget.

- Tekst = widget
- Knop = widget
- Pagina = widget

Wij maken nu onze eigen widget: ****Task****.

Waarom?

Omdat elke taak (todo-item) er hetzelfde uitziet:

- checkbox
- tekst

Als je dat 10 keer moet maken, wil je niet 10 keer dezelfde code typen.

Plak de `Task` widget **onderaan** in `main.dart` (onder `TodoHomePage`).

```
// CHANGE: nieuwe widget voor 1 taak
class Task extends StatelessWidget {
  const Task({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('Ik ben een Task widget');
  }
}
```

Nu moeten we de `Task` ook **gebruiken**. Vervang de `body` van `TodoHomePage` door:

```
// CHANGE: we gebruiken Task in plaats van losse tekst
body: const Center(
  child: Task(),
),
```

✓ Run weer. Je ziet nu: “Ik ben een Task widget”.

Stap 5 — Checkbox + label toevoegen (alleen widgets, nog geen logica)

Nu bouwen we de echte vorm van een taak.

Wat gaan we bouwen?

- Links: `Checkbox`
- Rechts: `Text`

Vervang de `build()` in `Task` door dit:

```
@override
Widget build(BuildContext context) {
  return Row(
```

```

children: const [
  // CHANGE: checkbox widget (nog niet klikbaar)
  Checkbox(
    value: false,
    onChanged: null, // null = uitgeschakeld
  ),

  // CHANGE: label/tekst widget
  Text('Voorbeeld taak'),
],
);
}

```

Uitleg

- `Row(...)` zet widgets **naast elkaar**.
- `Checkbox(value: false)` betekent: hij staat uit.
- `onChanged: null` betekent: je kunt er nog niet op klikken.

✓ Run weer. Je ziet nu een checkbox + tekst.

Stap 6 — Waarom state nodig is (Task moet “onthouden” of hij aan/uit staat)

Het probleem

We willen:

- checkbox aan klikken
- checkbox blijft aan staan

Maar...

- Onze `Task` is nu een **StatelessWidget**
- “Stateless” betekent: **onthoudt niets**

Dus we moeten **State** gebruiken.

Oplossing

We maken `Task` een **StatefulWidget**.

Vervang de hele `Task` class door dit:

```
// CHANGE: Task is nu StatefulWidget i.p.v. StatelessWidget
class Task extends StatefulWidget {
  const Task({super.key});

  @override
  State<Task> createState() => _TaskState();
}

class _TaskState extends State<Task> {
  // CHANGE: dit is state (data die we willen onthouden)
  bool isDone = false;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Checkbox(
          // CHANGE: checkbox leest nu uit state
          value: isDone,
          onChanged: null, // nog steeds niet klikbaar
        ),
        const Text('Voorbeeld taak'),
      ],
    );
  }
}
```

Belangrijk idee

- `isDone` is een variabele die **in memory** staat.
- Flutter kan die gebruiken om opnieuw te tekenen.

✓ Run. De checkbox ziet er hetzelfde uit, maar nu hebben we state.

Stap 7 — Checkbox togglebaar maken met state + event

Nu maken we de checkbox klikbaar.

Wat is een event?

Een event is iets dat gebeurt, zoals:

- iemand klikt
- iemand tapt

- iemand typt

Bij een checkbox heet dat event: `onChanged` .

Pas de `Checkbox` aan naar:

```
Checkbox(  
  value: isDone,  
  // CHANGE: onChanged krijgt een functie (dus niet null)  
  onChanged: (newValue) {  
    // newValue is wat de checkbox wil worden (true/false)  
  
    // CHANGE: setState vertelt Flutter: "teken opnieuw"  
    setState(() {  
      isDone = newValue ?? false;  
    });  
  },  
)
```

Waarom `setState` ?

Zonder `setState` verandert `isDone` misschien wel, maar Flutter weet niet dat hij het scherm opnieuw moet tekenen.

✅ Run. Nu kun je de checkbox aan/uit zetten.

Stap 8 — Meerdere Task widgets met een JSON datastructuur

Nu wordt het leuk: in plaats van 1 taak willen we er meerdere.

8.1 — Wat bedoelen we met “JSON datastructuur”?

Echte JSON ziet er zo uit (bijvoorbeeld vanuit een API):

```
[  
  {"title": "Boodschappen doen", "done": false},  
  {"title": "Huiswerk maken", "done": true}  
]
```

In Dart maken we iets dat hierop lijkt:

- Een **lijst** (`List`)
- Met daarin **maps** (`Map<String, dynamic>`)

Waarom `dynamic` ?

- Omdat de value soms `String` is (title)
- en soms `bool` is (done)

8.2 — We maken de home page Stateful (zodat we een lijst kunnen bewaren)

We gaan `TodoHomePage` ombouwen naar `StatefulWidget`, zodat hij taken kan bewaren.

Vervang `TodoHomePage` helemaal door:

```
// CHANGE: TodoHomePage is nu StatefulWidget
class TodoHomePage extends StatefulWidget {
  const TodoHomePage({super.key});

  @override
  State<TodoHomePage> createState() => _TodoHomePageState();
}

class _TodoHomePageState extends State<TodoHomePage> {
  // CHANGE: "JSON-achtige" lijst met taken
  final List<Map<String, dynamic>> tasksJson = [
    {'title': 'Boodschappen doen', 'done': false},
    {'title': 'Huiswerk maken', 'done': true},
    {'title': 'Flutter oefenen', 'done': false},
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Todo-app'),
      ),

      // CHANGE: straks komt hier ListView.builder
      body: const Center(
        child: Text('Straks renderen we een lijst...'),
      ),
    );
  }
}
```

✓ Run. Je ziet even weer tekst, maar nu hebben we `tasksJson` klaarstaan.

8.3 — Meerdere items renderen met `ListView.builder`

Een `ListView` is een scrollbare lijst.

Waarom `builder`?

- Omdat Flutter dan **per item** een widget maakt
- Dat is efficiënt (zeker bij grote lijsten)

Vervang de `body` in `_TodoHomePageState` door:

```
// CHANGE: lijst maken met builder
body: ListView.builder(
  itemCount: tasksJson.length,
  itemBuilder: (context, index) {
    // CHANGE: 1 map (1 taak) pakken uit de lijst
    final taskMap = tasksJson[index];

    // CHANGE: Task widget tonen (voor nu nog zonder data)
    return const Padding(
      padding: EdgeInsets.all(8.0),
      child: Task(),
    );
  },
),
```

✓ Run. Je ziet nu meerdere keren dezelfde “Voorbeeld taak”.

8.4 — Data uit JSON doorgeven aan de Task widget

Nu komt een belangrijk stukje Flutter-denken.

Tot nu toe was onze `Task` widget altijd hetzelfde. Maar nu willen we dat elke `Task` **andere data krijgt**.

Bijvoorbeeld:

- Task 1 → "Boodschappen doen"
- Task 2 → "Huiswerk maken"
- Task 3 → "Flutter oefenen"

Dat betekent: 🙌 De `Task` widget moet informatie **van buitenaf** krijgen.

In Flutter doe je dat via de **constructor (parameters)** van een widget.

Eerst even een simpel voorbeeld (zonder Flutter)

Stel je hebt deze Dart class:

```
class Persoon {  
  final String naam;  
  
  Persoon(this.naam);  
}
```

En je maakt er eentje:

```
Persoon p = Persoon("Mark");
```

Wat gebeurt hier?

- We geven "Mark" mee
- De constructor stopt dat in `naam`
- De class onthoudt dat

Widgets werken precies zo.

Wat willen wij?

Wij willen dit kunnen doen:

```
Task(  
  title: "Boodschappen doen",  
  initialDone: false,  
)
```

Dus:

- `title` → tekst van de taak
- `initialDone` → beginstand van de checkbox

Maar daarvoor moet `Task` eerst leren om die waarden te accepteren.

Stap 1 — Task krijgt parameters

Vervang `class Task extends StatefulWidget` door:

```
// CHANGE: Task krijgt data mee: title + initialDone
class Task extends StatefulWidget {
  const Task({
    super.key,
    required this.title,          // CHANGE: verplicht veld
    required this.initialDone,    // CHANGE: verplicht veld
  });

  // CHANGE: dit zijn eigenschappen van de widget
  final String title;
  final bool initialDone;

  @override
  State<Task> createState() => _TaskState();
}
```

Wat gebeurt hier precies?

1 final String title;

Dit is een **variabele van de widget zelf**.

Elke Task heeft dus zijn eigen `title`.

2 required this.title

Dit betekent:

- Als je een `Task()` maakt
- Dan MOET je een `title` meegeven
- Anders krijg je een foutmelding

Dat is expres. Wij willen niet dat iemand per ongeluk een lege Task maakt.

3 Waarom final ?

Omdat:

- Een widget zelf verandert niet
 - Alleen de **State** verandert
-

Stap 2 — Hoe komt de data in de State terecht?

Nu hebben we dit probleem:

- `title` zit in de Widget
- `initialDone` zit in de Widget
- Maar onze checkbox gebruikt `isDone` in de **State**

Dus we moeten bij het starten van de State de waarde overnemen.

Dat doen we in `initState()`.

Pas `_TaskState` aan naar:

```
class _TaskState extends State<Task> {
  // CHANGE: we maken het late, want we vullen hem in initState
  late bool isDone;

  @override
  void initState() {
    super.initState();

    // CHANGE: startwaarde komt uit de Widget
    // widget.initialDone betekent:
    // "pak de waarde uit de bovenliggende Task widget"
    isDone = widget.initialDone;
  }

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Checkbox(
          value: isDone,
          onChanged: (newValue) {
            setState(() {
              isDone = newValue ?? false;
            });
          },
        ),

        // CHANGE: tekst komt nu uit widget.title
        Text(widget.title),
      ],
    );
  }
}
```

Wat is `widget.title` eigenlijk?

In een `State` class heb je automatisch toegang tot:

```
widget
```

Dat verwijst naar de bijbehorende `Widget`.

Dus:

- `widget.title` → de title die werd meegegeven
- `widget.initialDone` → de beginwaarde

Dat is typisch Flutter. En even wennen 😊

Stap 3 — De data echt meegeven vanuit `ListView`

Nu gaan we in `ListView.builder` dit doen:

```
// CHANGE: Task krijgt title + initialDone uit taskMap
return Padding(
  padding: const EdgeInsets.all(8.0),
  child: Task(
    title: taskMap['title'] as String,
    initialDone: taskMap['done'] as bool,
  ),
);
```

Wat gebeurt hier stap voor stap?

1 `taskMap = tasksJson[index]`

We pakken 1 item uit de lijst.

Bijvoorbeeld:

```
{'title': 'Huiswerk maken', 'done': true}
```

2 `taskMap['title']`

Dat betekent:

👉 Pak de value die hoort bij de key `'title'`

Dat is een `String`.

3 `as String`

Waarom doen we dat?

Omdat Dart zegt:

- De map is `dynamic`
- Dus ik weet niet zeker welk type dit is

Met `as String` zeggen wij: 👉 "Vertrouw me, dit is een String"

4 Wat gebeurt er dan in Flutter?

Voor elk item in de lijst:

Flutter doet eigenlijk dit:

```
Task(title: "Boodschappen doen", initialDone: false)
Task(title: "Huiswerk maken", initialDone: true)
Task(title: "Flutter oefenen", initialDone: false)
```

Dus:

- Elke Task krijgt zijn eigen data
- Elke Task heeft zijn eigen State
- Elke checkbox werkt onafhankelijk

Dat is super krachtig 💪

Belangrijk concept om te onthouden

In Flutter:

- Data gaat **van boven naar beneden** (parent → child)
- Een widget krijgt data via **constructor parameters**
- State gebruikt die data via `widget.variabele`

Als je dit snapt... Dan snap je al een groot deel van Flutter 🎉

✓ Run.

- Je ziet nu verschillende titels
- Sommige checkboxes staan al aan (bijv. “Huiswerk maken”)
- Elke checkbox werkt los van elkaar

En dat allemaal zonder database 🔥

Eindresultaat — Hele `main.dart` (alles bij elkaar)

Dit is de complete versie. Handig als je wilt checken of je alles goed hebt.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const TodoApp());
}

class TodoApp extends StatelessWidget {
  const TodoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: TodoHomePage(),
    );
  }
}

class TodoHomePage extends StatefulWidget {
  const TodoHomePage({super.key});

  @override
  State<TodoHomePage> createState() => _TodoHomePageState();
}

class _TodoHomePageState extends State<TodoHomePage> {
  final List<Map<String, dynamic>> tasksJson = [
    {'title': 'Boodschappen doen', 'done': false},
    {'title': 'Huiswerk maken', 'done': true},
    {'title': 'Flutter oefenen', 'done': false},
  ];

  @override
  Widget build(BuildContext context) {
```

```

        appBar: AppBar(
          title: const Text('Todo-app'),
        ),
        body: ListView.builder(
          itemCount: tasksJson.length,
          itemBuilder: (context, index) {
            final taskMap = tasksJson[index];
            return Padding(
              padding: const EdgeInsets.all(8.0),
              child: Task(
                title: taskMap['title'] as String,
                initialDone: taskMap['done'] as bool,
              ),
            );
          },
        ),
      );
    }
  }
}

```

```

class Task extends StatefulWidget {
  const Task({
    super.key,
    required this.title,
    required this.initialDone,
  });

  final String title;
  final bool initialDone;

  @override
  State<Task> createState() => _TaskState();
}

```

```

class _TaskState extends State<Task> {
  late bool isDone;

  @override
  void initState() {
    super.initState();
    isDone = widget.initialDone;
  }

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Checkbox(
          value: isDone,
          onChanged: (newValue) {

```



```
        setState(() {  
            isDone = newValue ?? false;  
        });  
    },  
    ),  
    Text(widget.title),  
  ],  
);  
}  
}
```